Acuo VNA

Acuo Deidentification Service User Guide

Version 25.1

February 2026

© Hyland Software, Inc. All rights reserved.

Table of Contents

Acuo Deidentification Service Overview 3
Installation and Upgrades 3
New Installation Steps 3
Upgrade Steps 4
Configuration Properties 4
Modes and Options 5
Deidentification 5
Decoupled Mode 8
Bulk Mode 8
Redaction 8
Part 10 Only 9
Get Uncompressed P10 image 10
DIR Option 11
DICOM Standard Supported Profiles 11
Deidentification Profiles 11
ExtendedTags – Private Tags 12
AnonymizationActions 13
Hashing 13
Substituted Values 14
TimeShiftTag 14
DICOM SCP Functionality 14
RESTful API 16
Context Management Operations 17
Email Configuration 17
Job ID 18
Content Saving Operation 18
Annex A – API 19
Annex B – Expanded API 21
Deidentification Service FAQs 21

© Hyland Software, Inc. All rights reserved.

Acuo Deidentification Service Overview

The Acuo Deidentification Service is a component provided as a Windows Service (AcuoDeidentification) and deployed via an installer.

It works in conjunction with the 2018 Q1 or newer versions of AcuoAccess.

The Deidentification service is being driven via input files, which can be dropped into an input watch directory, or via a RESTful API. While we are still calling the service Deidentification it can also drive the generation of original Part10 files written to a preconfigured partition or a partition specified as part of a RESTful POST analytics request. Analytics requests have evolved, for example allowing metadata to be indexed by a search engine.

Installation and Upgrades

New Installation Steps

1. Run the "AcuoDeidentificationSetup.msi" file.
2. On the Welcome screen, select Next.
3. On the License Agreement screen, check the box to accept the License Agreement and select Next.
4. On the Destination folder screen that indicates the location where files will be installed, select “Next” (or change the directory as needed and select Next).
5. On the Acuo Deidentification Configuration Settings screen, make any necessary changes and then select Next:
    a. ***Part10 WebService Server and Part10 WebService Port***: Enter these as defined from AcuoAccess.
    b. ***Deidentification Directory***: Location for deidentification output.
    c. ***Part10 Directory***: Location for storing imported images using the AcuoAccess REST call.
    d. ***History Directory***: Location for capturing historical data.
    e. ***Context Directory***: Location of deidentification context files, which contain the details of the study and the deidentified data (file name format = context_jobid.deident).
    f. ***Input Watch Directory***: A drop directory for the input files that contain studies to be deidentified.
6. On the Acuo Deidentification Database Settings screen, provide the AcuoMed Database connection details and then select Next:
    a. ***Database Server Name or IP***: Enter server name or IP where AcuoMed Database resides.
    b. ***Initial database catalog name***: Name of the AcuoMed Database (usually left at default AcuoMed).
    c. ***Login database user***: Leave as-is (integrated).
    d. ***Password database user***: Enter password for the AcuoMed Database.
7. Select “Install” on the following screen to proceed with the installation, then select “Finish” when the installer completes to exit.

© Hyland Software, Inc. All rights reserved.

Note: After the installation completes, the configuration settings and properties from the installer can be modified in the following primary configuration file:

- **\Program Files\Acuo Technologies\AcuoDeidentification\ AcuoDeidentificationService.exe.config**

Reference the AcuoDeidentification Configuration Properties section below.

Upgrade Steps

Upgrading consists of the following steps:

1. Stop the AcuoDeidentification Service.
2. Backup AcuoDeidentification files, including the primary configuration file:
    a. \Program Files\Acuo Technologies\AcuoDeidentification\AcuoDeidentificationService.exe.config
3. Install AcuoDeidentification using the msi installer file (AcuoDeidentificationSetup.msi).
4. Replace the AcuoDeidentificationService.exe.config at \Program Files\Acuo Technologies\AcuoDeidentification from the backup file.
5. Start the AcuoDeidentification Service.

Configuration Properties

The Acuo Deidentification service is being configured by a .NET style app.config file, located at the following directory:

\Program Files\Acuo Technologies\AcuoDeidentification**AcuoDeidentificationService.exe.config**

```
<add key="DBConnectionString" value="Integrated Security=true;Initial Catalog=AcuoMed;Data Source=DBServer" />

<add key="Part10Directory" value="c:/temp/part10"/>
<add key="DeidentificationDirectory" value="c:/temp/deidentify"/>
<add key="ContextDirectory" value="c:/temp/deidentify/context"/>
<add key="InputWatchDirectory" value="c:/temp/deidentify/watch"/>
<add key="Part10WebServiceEndpoint" value="http://localhost:8091/AcuoREST/genp10"/>
<add key="HistoryDirectory" value="c:/temp/deidentify/history"/>

<add key="DeidentificationMaxDOP" value="1"/>

<add key="IgnoreHistory" value="false"/>

<add key="BulkMode" value="false"/>

<add key="DecoupledMode" value="false"/>

<add key="RestServiceAddress" value="http://0.0.0.0:8099/AcuoDeidentification" />
```

© Hyland Software, Inc. All rights reserved.

```
<add key="RemoteDicomDestination" value="TestDest2:localhost:1142"/>
```

DBConnectionString – is needed to connect to the VNA's database for study identification

IgnoreHistory – true or false, controls if HistoryDirectory will be ignored

Part10WebServiceEndpoint – the prefix of the AcuoAccess web service for Part10 generation

Part10Directory – partition where the study is written in Part10

DeidentificationDirectory – partition where the deidentified study is written

ContextDirectory – contains the deidentification context file, name context.deident

HistoryDirectory – contains the information on successfully processed files

InputWatchDirectory – drop directory for the input files that contain studies to be deidentified

DecoupledMode – in this mode, the RESTful query service and WADO will be used, and the deidentification service can reside on a server that has only access to the AcuoAccess endpoint

RestServiceAddress – the endpoint for the RESTful API, which can drive deidentification requests

BulkMode – in this mode no deidentification context will be used. This mode is ideal for high concurrency and large number of studies. It operates at the patient level and with entries of type MRNBLK

DeidentificationMaxDOP – study deidentification in BulkMode (=true) can be performed with a degree of parallelism specified by the value of this property

RemoteDicomDestination – global DICOM destination, all jobs will be sent to this destination, the programmatic API non-null values overwrite this value. Its format is AETitle:Host:Port

LocalAETitle – Local AETitle value used for DICOM send operations, default value "Deidentification".

DoNotDeleteAfterSend – If the property value is true, the data will not be deleted after a successful send to a DICOM destination. The default value is false, and the default behavior is to delete the deidentified data from the output directory in the case of a successful transmission of the study.

Modes and Options

Deidentification

In the case of the input watch directory, the input file consists of a collection of lines, each line identifying a DICOM study. Multiple ways exist to identify a study. Each line is a list of fields, separated by the '|' character. The first field is the type of identification, which is one of 'OID', 'ACC', 'MRNSTD' and 'MRNBLK' (for the bulk option).

For 'OID'

OID|StudyInstanceUID

© Hyland Software, Inc. All rights reserved.

or

OID|StudyInstanceUID|Database

are valid formats. Specifying the database is preferred since it will trigger only one SQL query, and it refers to the Acuo DICOM database the study is part of. ‘OID’ is the preferred way for study identification.

For ‘ACC’

ACC|AccessionNumber

or

ACC|AccessionNumber|Database

are valid formats.

For ‘MRNSTD’

MRNSTD|PatientID|StudyDate

or

MRNSTD|PatientID|StudyDate|Database

are valid formats.

For ‘MRNBLK’

MRNBLK|PatientID

or

MRNBLK|PatientID|Database

are valid formats.

For ‘ACCBLK’

ACCBLK|AccessionNumber

or

ACCBLK|AccessionNumber|Database

are valid formats.

This input “MRNBLK” or “ACCBLK”, should be used only in correlation with the BulkMode option.

If the information provided for a study finds more than one study, all studies will be deidentified.

The Deidentification service will compute each study entry, by running a query against the VNA’s database to identify the study or studies.

© Hyland Software, Inc. All rights reserved.

Once the study is identified, a web service call to AcuoAccess will be made to generate the Part10 study on disk. The Web Service call will need the PatientID, StudyInstanceUID and database of the study, which are obtained during the study identification step regardless what type of identification is used.

http://{host}:{port}/AcuoREST/genp10/domains/{Database}/patients/{PatientID}/studies/{StudyInstanceUID}

Above, you will find the web service endpoint. The genp10 web service call is synchronous, and blocks until the data has been written to disk. AcuoAccess will write the study in a directory specified by property PreviewPostFetcherPart10Directory. The study will be written in a directory format that contains the Database, PatientID, and StudyInstanceUID subdirectories, and each DICOM instance will be a Part10 file with the SOPInstanceUID as its name and the extension .dcm.

Excluding DICOM instances part of a study from being generated as part of a Part10 request can be done at the modality or SOPClass level and is controlled by properties Part10SOPClassExclusions and Part10ModalityExclusions. The values are comma separated lists. Both properties can be defined at the same time.

Once the study Part10 generation is complete, the deidentification operation will process the study from the same directory location and write the deidentified study to an output directory in a similar directory format. Please note that the PatientID, StudyInstanceUID and SOPInstanceUID values will be changed by the deidentification operation. Once completed, the Part10 study will be deleted.

A history directory is used to track the studies already processed. These will be skipped in case a run is interrupted and resumed. The history directory will have to be cleared in case this is not intended. Taking the history directory into consideration can be skipped via configuration (property IgnoreHistory).

Another important data point is the anonymization context, which is being persisted between the anonymization of each study (save frequency can be controlled), and will be written into the context directory.

The anonymization values for PatientID, StudyID, and other ID fields, are using a counter based system. The seed of the counter can be set via the property

```
<setting name="IndexSeed" serializeAs="String">
        <value>1000</value>
</setting>
```

of the <Hyland.Nil.Deidentify.Properties.Settings> section.

This will generate PatientID values like [Patient ID.1001], [Patient ID.1003], etc.

If property AnonymizationRecordDirectory is defined, an anonymization report file will be created for each anonymization run. For each study there will be one entry in the file, which will help organizations managing their anonymization workflow.

OriginalPatientID|OriginalAccessionNumber|OriginalStudyInstanceUID|DeidentifiedPatientID|DeidentifiedAcccessionNumber|DeidentifiedStudyInstanceUID

In the same <Hyland.Nil.Deidentify.Properties.Settings> section, property ReidentificationSecret, which is encrypted, will be used as a key to encrypt the content of the EncryptedAttributeSequence.

© Hyland Software, Inc. All rights reserved.

Decoupled Mode

The Acuo Deidentification service has a decoupled mode as well. In this mode, triggered by property DecoupledMode being set to true (default is false), the service will use the RESTful query service and WADO classic requests from AcuoAccess to identify the studies and then retrieve them in Part10 format to the Part10Directory. In this mode, the Deidentification service does not need to be “close” to the VNA and have access to a shared Part10Directory partition. It also doesn’t need to have access to the VNA’s database server. It can reside on a remote server, with the only requirements being access to the AcuoAccess web service endpoint for search and WADO, and having enough space on the Part10Directory partition to save the files. The WADO calls are being made with the AcuoAccess specific optimizations of specifying the domainID, which is the Acuo DICOM database the study resides in and transferSyntax=donottranscode, which will retrieve the study with the same transfersyntax it resides on the VNA.

Bulk Mode

This mode doesn’t use a deidentification context and is suited for large collections of studies and allows high concurrency. It is triggered by property BulkMode being set to true. DecoupledMode needs to be set to false.

The input file for this mode needs to be composed of entries of type MRNBLK. This mode will operate at the Patient level. This is necessary in order to eliminate reliance on the deidentification context. We also strongly recommend the usage of a pattern for person names like

```
<setting name="PseudonymPattern" serializeAs="String">
        <value>TEST{0}^ANONYMOUS^^^</value>
</setting>
```

Redaction

Redaction – adding a noise pattern to mask burned in demographics – is based on the same anonymization library. It is by default turned off however can be controlled via the following property

```
<setting name="Redact" serializeAs="String">
        <value>True</value>
</setting>
```

of the <Hyland.Nil.Deidentify.Properties.Settings> section.

Applying redaction is based on templates, primarily identifying the modality, manufacturer and model number, as well as matching rows and columns data element values. A template will specify the rectangle used by the noise pattern. The property defining the settings is Masks.

```
<setting name="Masks" serializeAs="String">
          <value>&lt;Masks&gt;
&lt;DS x00080060="US" x00080070="Philips Medical Systems" x00081090="iE33" x00280010="600"
x00280011="800"&gt;
```

© Hyland Software, Inc. All rights reserved.

<RG top="6" left="410" width="190" height="18" />
<RG top="34" left="4" width="100" height="18" />
</DS>
<DS x00080060="US" x00080070="ATL" x00081090="HDI5000/DVS" x00280010="476" x00280011="640">
<RG top="0" left="40" width="180" height="16" />
<RG top="0" left="245" width="180" height="16" />
<RG top="0" left="430" width="90" height="32" />
</DS>
<DS x00080060="US" x00080070="ACUSON" x00081090="SEQUOIA" x00280010="480" x00280011="640">
<RG top="0" left="26" width="289" height="27" />
</DS>
<DS x00080060="US" x00080070="ACUSON" x00081090="SEQUOIA" x00280010="576" x00280011="768">
<RG top="0" left="0" width="768" height="37" />
</DS>
</Masks>
</value>
</setting>

Part 10 Only

There have been requests to generate only data in Part10 format in the configured partition. In this case, there will be no deidentification, the data will be obtained either in decoupled mode via WADO calls or the calls to the genP10 web services. The data will be left in the configured Part10 directory and not deleted, therefore enough space for the projected operations should be made available.

The input file is very similar to the main deidentification option and consists of a collection of lines, each line identifying a DICOM study. Multiple ways exist to identify a study. Each line is a list of fields, separated by the '|' character. The first field is the type of identification, which is one of 'OID_P10', 'ACC_P10', 'MRNSTD_P10' and 'MRNBLK_P10' (for the bulk option).

For 'OID_P10'

OID_P10|StudyInstanceUID

or

OID_P10|StudyInstanceUID|Database

are valid formats. Specifying the database is preferred since it will trigger only one SQL query, and it refers to the Acuo DICOM database the study is part of. 'OID_P10' is the preferred way for study identification.

For 'ACC_P10'

ACC_P10|AccessionNumber

or

ACC_P10|AccessionNumber|Database

© Hyland Software, Inc. All rights reserved.

are valid formats.

For ‘MRNSTD_P10’

MRNSTD_P10|PatientID|StudyDate

or

MRNSTD_P10|PatientID|StudyDate|Database

are valid formats.

For ‘MRNBLK_P10’

MRNBLK_P10|PatientID

or

MRNBLK_P10|PatientID|Database

For ‘ACCBLK_P10’

ACCBLK_P10|AccessionNumber

or

ACCBLK_P10|AccessionNumber|Database

are valid formats. are valid formats. This MRNBLK_P10 and ACCBLK_P10should be used only in correlation with the BulkMode option.

This paragraph is somewhat related to the DeIdentification service. If there are direct integrations with the genp10 web services endpoints of AcuoAccess, the static configuration value of PreviewPostFetcherPart10Directory can be set per request by using the value of header, "Acuo-TargetDirectory". This can be specified programmatically or with a tool like curl. In a few situations, a Job ID may also be relevant for the genp10 request. This can be set via the value of header "Acuo-Analytics-JobID".

Get Uncompressed P10 image

If the exams are stored compressed in the VNA source (Jpeg Lossless or Jpeg 2000 Lossless) and P10 request is made, the p10 data is written/obtained in the same compressed format.

But, if user wants to get the P10 data in uncompressed format (ILE/Implicit VR Endian/1.2.840.10008.1.2 or ELE/Explicit VR Little Endian/1.2.840.10008.1.2.1), the below mention configuration keys should be added in the config file –

```
<add key="DownloadUncompressedP10Images" value="true" />
<add key="P10UncompressedTransferSyntaxValue" value="1.2.840.10008.1.2" />
```

© Hyland Software, Inc. All rights reserved.

DIR Option

Files dropped in the watch directory can support the format

DIR|InputDirectory|OutputDirectory

In this case, the deidentification is going to collect all the ".dcm" files in the InputDirectory, assumed in Part10 and deidentifies these using the OutputDirectory as the target. This option parses the InputDirectory recursively. The filenaming strategy used for output is the same "DeIdPatientID\DeIdStudyInstanceUID\DeIdSOPInstanceUID.dcm".

For example

DIR|C:\temp\part10\212121\1.2.840.113619.2.30.1.1762288588.1465.910693725.342|C:\temp\deidentify\output

DICOM Standard Supported Profiles

A new property has been added to the app.config

```
<!--<add key="Profiles"
value="RetainSafePrivate,RetainUIDs,RetainDeviceIdent,RetainInstitutionIdent,RetainPatientChars,RetainLongFullDates,RetainLongModifDates,CleanDesc,CleanStructCont,CleanGraph" />-->
```

All supported options are in the string above, please consult with DICOM PS 3.15 and remember that some options are conflicting.

An example is to trigger the "Retain Longitudinal Temporal Information With Modified Dates Option".

```
<add key="Profiles" value="RetainLongModifDates" />
```

Deidentification Profiles

Due to multiple feedback opportunities, it has been recognized that a deployment of the Deidentification Service will have to support more than one research integration in large institutions. Therefore, support for multiple deidentification profiles has been added. Think of different configurations per deployment identified by a profile id, which is a string.

```
<DeidentificationProfileConfigSection>
   <DeidentificationProfiles>
     <DeidentificationProfile ProfileId="ResearchGroup1" DeidentificationEpochs="1960"
TimeShiftTag="0x00100030"/>
     <DeidentificationProfile ProfileId="PartnerProfile2" AnonymizationActions="[{&quot;Tag&quot;:
&quot;(0008,0050)&quot;,&quot;BasicProfile&quot;: &quot;Z&quot;}]"/>
   </DeidentificationProfiles>
  </DeidentificationProfileConfigSection>
```

The above app.config shows the definition of two deidentification profiles, identified by profile ids, ResearchGroup1 and PartnerProfile2. Below is an enumeration of the properties that can be defined per profile. If a property is not defined in a profile, it will be defined by the default property.

© Hyland Software, Inc. All rights reserved.

```
public class DeidentificationProfile
  {
    public string ProfileId { get; set; }
    public string ScrubberDefaultAllowed { get; set; }
    public string[] ScrubberDefaultUntouched { get; set; }
    public string ScrubberTrim { get; set; }
    public string PseudonymPattern { get; set; }
    public string PseudonymPrefix { get; set; }
    public string PseudonymSuffix { get; set; }
    public string PseudoidFormat { get; set; }
    public string PseudoDefaultFormat { get; set; }
    public string AnonymizationExtensions { get; set; }
    public string AnonymizationActions { get; set; }
    public string ExtendedTags { get; set; }
    public string DicomUidRoot { get; set; }
    public string ReidentificationSecret { get; set; }
    public int IndexSeed { get; set; }
    public int ContextSaveFrequency { get; set; }
    public int TimeShiftTag { get; set; }
    public string[] DeidentificationEpochs { get; set; }
    public string[] DicomDeidProfiles { get; set; }
    public bool HashedValues { get; set; }
    public ulong HashSalt { get; set; }
    public DirectoryInfo DeidOutputDirectory { get; set; }
    public DirectoryInfo ReidOutputDirectory { get; set; }
    public DicomDestinationInternal DicomDestination { get; set; }
    public DicomDestinationInternal ReidDicomDestination { get; set; }
    public string StowCredentials { get; set; }
    public string StowReidCredentials { get; set; }
    public string DeidentificationSCP { get; set; }
    public string ReidentificationSCP { get; set; }
    public string SearchCollection { get; set; }
    public bool SubstitutedValues { get; set; }
    public Dictionary<string, Dictionary<string, string>> TagSubstitutions { get; set; }

  }
```

The ProfileId is a string, and in most cases, it will be submitted via a programmatic RESTful POST request, however it can also be specified as part of the name of a file dropped into the watch directory (as the second component after the Job ID), please refer to the Job ID section.

Hashing, SubstitutionValues and Anonymization actions will be explained in subsequent paragraphs.

ExtendedTags – Private Tags

Private tags are typically removed during the de-identfication process, except for the RetainSafePrivate profile option. To treat private tags as regular tags, these need to be added as extensions to the DICOM dictionary. This is achieved via property ExtendedTags.

```
<setting name="ExtendedTags" serializeAs="String">
        <value>
         {"0x00330030":[{"tag":"0x00333001","name":"", "vr":"LO"}, {"tag":"0x00333002","name":"",
"vr":"CS"}],"0x00330040":[{"tag":"0x00334001","name":"", "vr":"LO"},{"tag":"0x00334002","name":"",
"vr":"CS"}],"0x37110010":[{"tag":"0x3711100C","name":"", "vr":"UI"}]}
        </value>
```

© Hyland Software, Inc. All rights reserved.

If ExtendedTags is specified as part of a deidentification profile in the DeidentificationProfileConfigSection, all the “ characters will be needed to be substituted with ", for successful de-serialization – it is a combination of a JSON string in an XML style property config.

The ExtendedTags structure is a dictionary, with the key being the private creator tag and the values an array of the private tags. The name property is optional, the tag and vr fields are required.

Once defined as ExtendedTags, these private tags can be used as AnonymizationExtensions or part of AnonymizationActions.

AnonymizationActions

In some situations, the users want to be in full control of the de-identification actions, decoupling these from the actions defined by the DICOM standard’s Attribute Confidentiality Profiles.

If property AnonymizationActions is not empty or null, only the actions defined by this property will be applied to the DICOM instances.

For example

AnonymizationActions="[{"Tag": "(0008,0050)","BasicProfile": "Z"}]"

To make this easier to read, we have substituted " with “.

This simplified example means that only tag 0x00080050 (Accession Number) will be applied action ‘Z’, which is defined as *Z - replace with a zero length value, or a non-zero length value that may be a dummy value and consistent with the VR*.

AnonymizationActions="[{“Tag”: “(0008,0050)”,”BasicProfile”: “Z”}]"

Hashing

When Boolean property HashedValues is true, hashing algorithms will be applied to the string VRs, DA/DT VRs, PN VRs and UI VRs. At the core, CRC64 is used however the pre and post processing of values is more sophisticated. Hashed values correspond to 16 character strings, ‘0’-‘9’, ‘A’-‘F’, basically hexadecimals strings. SH values will always be 16 characters, LO VRs, 32 characters, a combination of two CRC64 values. In the case of UI VRs, the strings will be compliant and for dates, hashing is used for members of the TimeMachine class which is responsible for DA and DT shifting.

The hashed values cannot be reversed. A non-zero unsigned int value should be used for property HashSalt for each profile that is using HashedValues. This is similar to an initialization vector used by symmetric encryption algorithms. If different HashSalt values are used, the hashed results will be different. Using HashedValues true and the same value for the HashSalt, will ensure that the same values will be generated for anonymized values every time deidentification is applied to the DICOM dataset, which opens interesting consistency, performance, and scalability options.

With a hashing approach, since the DICOM VRs are typically shorter strings, collisions are a risk, because a larger value space is mapped into a smaller hashed value space. We do have a tool that

© Hyland Software, Inc. All rights reserved.

evaluates different DICOM value types stored in a VNA. So far, collisions have not been an issue for entire data set values stored in large VNA systems. This is because collisions are not that common and because the pre and post processing actions are more complex than a simple CRC64 computation.

Substituted Values

For many clinical trial scenarios, it makes sense for certain tag substitutions to be specified by the client/driver of the deidentification service. For example, PatientIDs/MRNs. Substitution Values can be specified either by the Boolean SubstitutedValues property part of a deidentification profile being true, and a dictionary of values, TagSubstitutions. Or the more common scenario, it can be programmatically specified by defining the dictionary of substitution values as part of an AnalyticsRequest or DeIdentificationRequest and non-null member SubstitutionValues of the POST POCO submission object.

The ways to specify tag substitutions are tag|oldvalue|newvalue or tag||newvalue. The latter makes sense for CS VRs.

An example for a POCO POST submission is below.

{"Type":"OID","StudyEntries":[{"StudyInstanceUID":"1.2.840.113711.18102213.2.4064.307041100.25093968.110","Database":"DicomDB1"}],"ID":202105031814,"ProfileID":"ProfileResearch1","TargetDirectory":null,"Collection":null,"RemoteDicomDestination":null,"STOWURI":null,"StatusCallbackURI":null,"SubstitionValues":{"00100020":{"15454564":"SBST1234","ID5678":"SBST5678"},"00080050":{"A10012497374":"SBSTACC1234","ACC5678":"SBSTACC5678"}}}

This example falls into the tag|oldvalue|newvalue category, and substitutions are specified for a few PatientID (00100020) and AccessionNumber (00080050) values.

TimeShiftTag

If a valid DICOM tag value, which should be a DA VR, is specified as a value for profile property TimeShiftTag, then all the dates of a DICOM study will be shifted into the past or future using the value of this field as a base. This is also a use case driven by clinical trials. For the “whence” component, the property DeidentificationEpochs will be used.

DICOM SCP Functionality

A DICOM based pipeline can be defined for both de-identification, as well as re-identification. New properties have been added to the deidentification profile.

The ingestion of the data will be done by sending data to a DICOM SCP. Each profile can have a separate AETitle and port combination for de-identification as well as re-identification. Obviously, if re-identification of DICOM data is sought, the Encrypted Attribute Sequence must be present and the secret keys must match, otherwise exceptions will be thrown. The SCP defining properties are DeidentificationSCP and ReidentificationSCP. The format is AETitle:port.

© Hyland Software, Inc. All rights reserved.

```
<add key ="DeidentificationSCP" value="DEIDAE:1045"/>
<add key ="ReidentificationSCP" value="REIDAE:1045"/>
```

The AETitle and port and function (de-id or re-id) must be unique. Multiple Called AETitles (DICOM term) can be defined on the same port. The implementation will match it to the proper profile and function.

The data received over DICOM will be stored in a directory. The global properties are DeidentificationSCPDirectory and ReidentificationSCPDirectory. Think of these as the equivalent of the Part10 directory.

The processed data by will be initially stored in an output directory. The profile properties are DeidOutputDirectory (same as for non DICOM related functionality) and ReidOutputDirectory (new for DICOM re-id).

The DICOM ingestion capability decouples the de-identification functionality from the Acuo VNA as a source. Pipelines can be created that bypass the VNA entirely. Of course, VNA tenants can be destinations for de-identified or re-identified data.

For the de-identification functionality, the same profile DicomDestination property will apply. For the re-identification functionality, property ReidDicomDestination will define an optional external application entity to send the data to. The values of the profile properties mentioned above are in format LocalAE:RemoteAE:host:port.

The global properties for DICOM re-identification are LocalReidAETitle and RemoteReidDicomDestination. The global property for STOW is STOWREIDURI.

If STOW is used instead of DICOM there are different credentials for de-id external destinations and re-id external destinations, profile properties StowReidUserid and StowReidPassword. The global properties for STOW credentials are StowReidUserId and StowReidPassword.

The DICOM data received is stored in the SCPDirectory, subdirectories are the profile, the jobid and the studyinstanceuid. The DICOM files are stored with the *.dcm extension in the studyinstanceuid directory. The jobid is important for de-identification and is computed as a hash from the CallingAE and Remote Port, so that it identifies the DICOM association. The jobid can be programmatically specified if a positive integer number is specified as part of the Calling AETitle, the format is AE_jobid, for example if the C-Store SCU uses AE_37651 as its Calling AETitle, the jobID will be 37651.

Once the data received, the subdirectories will be checked periodically. An important global property is DicomCompletionIntervalSeconds, it applies to both de-id and re-id studies received over DICOM. Candidates for the next stage of the pipeline (de-id or re-id) must have a last write timestamp of their studyinstanceuid subdirectory that is older than the DicomCompletionIntervalSeconds. The default value is 2 minutes. This to ensure that the data transfer over an association has completed.

The check is done periodically and governed by global properties DeidentificationSCPCheckFrequency and ReidentificationSCPCheckFrequency, defined in seconds. Once the candidates are selected the study directories will be renamed by adding the suffix .deid or .reid. Once a batch has been selected, subsequent checks will be no-ops until the batch is complete.

When completed without exception, the suffix of the study directories will be changed to .deidc or .reidc.

The .deidc and .reidc directories can be cache cleaned if the DICOM SCP cache cleaner functionality is configured. This functionality is driven by several configuration properties.

© Hyland Software, Inc. All rights reserved.

DICOMSCPCacheCleanerIntervalSeconds is determining the frequency the cache cleaner is running. DICOMSCPCacheHoursToRetain is controlling which completion directories are selected for deletion, the last write timestamp most be older that the value of this property in hours. To expedite the cache cleaning operation a degree of parallelism can be configured for the directory deletion operation, controlled by DICOMSCPCacheCleanerDOP.

DICOM SCP de-identification and re-identification can have a higher degree of parallelism configured. The re-identification operations are decoupled, and the number of concurrent study re-identification operations can be tuned via property ReidentificationMaxDOP.

The de-identification operation takes advantage of the value of already introduced property DeidentificationMaxDOP. Due to complexity of context operations, the parallelism is applied at the jobid level since the de-identification contexts are separate.

RESTful API

The Acuo Deidentification service can be driven programmatically via a RESTful API. The endpoint is configured using the value of the RestServiceAddress configuration property. The API's method and POCO classes can be found part of the annexes of this document.

The main method is a POST request which takes a DeIdentificationRequest object. This class is composed of an array of StudyEntry objects, an ID, an optional DicomDestination object and a Status CallbackURI. The ProfileID is also very important. The StudyEntry is VNA specific and is a combination of a StudyInstanceUID and Database value fields. If non-null, the DicomDestination will be used, and after the deidentification operation, the studies will be sent to the remote DICOM destination specified by this object, and afterwards deleted. Otherwise (null DicomDestination) the study will remain on the preconfigured partition. This behavior (deletion) can be controlled by property DoNotDeleteAfterSend. If the value is true, the DICOM instances of the entry will also be left on the preconfigured output partition.

The IndexSeed can also be provided as part of a request. If the value is > 0, it will override the global IndexSeed value, or the one specified as part of the Deidentification Profile.

The StatusCallbackURI is used for the service to provide periodic status updates, at a configurable completion rate (default 25%), so that the completion of the deidentification request can be tracked in real-time. The Deidentification service is going to make an http PATCH request to the StatusCallbackURI, the POCO used is an instance of DeidentificationJobStatus.

There are also methods for obtaining a deidentification report for a specific job/deidentification request (GetDeidentificationReport – per job) and viewing of the changes applied between the original and deidentified data (GetStudyReport – per job and study).

An additional set of POCOs (Plain CLR objects) and APIs are available, which combine Deidentification and Part10 requests, into an AnalyticsRequest.

In addition to a DicomDestination, a STOWURI web service endpoint has been added and will send the studies over STOW-RS. If both the DICOMDestination and the STOWURI are non-null, DICOM will have precedence, and STOW will be ignored.

© Hyland Software, Inc. All rights reserved.

The Type of an AnalyticsRequest is a string and acceptable values are “DID” for deidentification requests and “P10” for Part10 requests.

The value of the TargetDirectory can be used in correlation with “P10” requests only, this will override the preconfigured partition value in AcuoAccess. The TargetDirectory will have to exist and the Hyland services will need to be run under accounts with permissions to write to it.

The Collection property is not used at this stage and is unrelated to types “DID” and “P10”.

Situations were identified where synchronous processing of a job submission was desired. For example, use cases that have small submissions sets and that are part of a more complex research pipeline or workflow.

Therefore, there is a version of the POST request that is synchronous and will block until the deidentification or analytics job is successful, or an exception is encountered.

The synchronous POST methods will return a POCO of type DeidentificationJobStatus.

Context Management Operations

A RESTful call is available for obtaining information about the various context objects. The GET operation will return an array of ContextInfo objects. The properties are the ID (1:1 relationship with the context object), the size on disk, the creation datetime and the last update datetime.

A DELETE call by ID is also available. This allows for unused contexts to be deleted. There are use cases where submissions separated in time can be made using the same JobID in order to leverage the persisted context information. This is not known to the application and the relationship and intent is obviously managed by the user of the deidentification service. These two calls can be used for creating watermarking functionality outside the application. An application embedded automated context cleaner can be implemented only for very old context objects, that haven’t been updated in a while.

Email Configuration

Email can be configured as a global setting. Whenever a status update PATCH call is made, an email with the same body as the PATCH request is being sent if the email properties are configured.

```
<add key="SmtpServer" value="smtp.gmail.com"/>
<add key="SmtpPort" value="587"/>
<add key="SmtpUser" value="Goj9225Nno7Ka2DuVl8kpzUgOa63kKR4A="/>
<add key="SmtpPwd" value="37Xvasf99Zwtg=="/>
<add key="SmtpUseSSL" value="true"/>
<add key="SmtpFromAddress" value="no_reply@acuodeidentification.com"/>
<add key="SmtpToAddresses" value="first.last@hyland.com,first2.last2@hyland.com"/>
```

The User and Pwd values are encrypted.

© Hyland Software, Inc. All rights reserved.

Job ID

The ID property of a DeidentificationRequest or AnalyticsRequest is the ID of the job. Other calls like status, anonymization report, and deidentification report (details of changed data) are requiring a job identifier. The deidentification context is also maintained for a job id.

There is also a way to specify a job ID when using the watch directory drop files. If the name of the file starts with the prefix 'xxxx_' where xxxx is an unsigned integer, this integer will be used as the job ID. For example, 523_CTProfile1_20200419.txt, in this case the Job ID for the entries in the file will be 523.

In the above example, CTProfile1 is the ProfileID, it is a string and the separator for the segments is '_', so 'xxxx_ProfileId_', where xxxx is the Job ID.

Based on this design, additional requests can be made spaced in time, with the same ID, if this is desired. This approach will work well for studies of the same patient and series of the same study, which will correctly attach to the DICOM Information Entity level based on the information persisted in the deidentification context. The management of IDs is done outside of the Deidentification service.

Please remember that the deidentification context can grow very large, so the size of the cumulative batches will need to be managed.

Content Saving Operation

As we have previously stated, the deidentification context can grow very large. The default implementation and configuration will save and reload the context for every single study. In most cases, exceptions are rare in the case of deidentification operations, so we added a new way to control the frequency of the save.

The ContextSaveFrequency will save the deidentification context only every n entries, value specified as part of the <Hyland.Nil.Deidentify.Properties.Settings> section, or a profile id. The default value is 1, and in line with the initial default behavior. Of course, this applies for BulkMode = false. The deidentification context is also saved when the batch of entries for deidentification has completed. Using a larger value for this property can significantly reduce the duration of the deidentification operation.

The only tradeoff is if the processing of the batches gets interrupted. In that case, the context saved on disk might not be in synch with the data in the output directory. This situation is rare, and the efficiency gains are significant in the case of large batches. To give you a more concrete example, if the batch size is 10,000 and a ContextSaveFrequency value of 50 is used, this will have the context persistence call performed only 200 times rather than 10,000 times. As the deidentification context grows – also dependent on the study type and size (number of DICOM instances) – the save operation becomes significant in duration (over a second), so the use of this configuration property is strongly encouraged for large batches and once the operators have confidence in the quality of their DICOM data. No exceptions will be thrown if no DICOM violations will be detected.

Again, this doesn't apply to BulkMode = true mode, which doesn't use a de-identification context.

```
<setting name="ContextSaveFrequency" serializeAs="String">
        <value>50</value>
</setting>
```

© Hyland Software, Inc. All rights reserved.

Annex A – API

```
[ServiceContract]
public class DeidentificationRestService
{

        [WebInvoke(Method = "POST", UriTemplate = "/deidentify", BodyStyle = WebMessageBodyStyle.Bare,
        RequestFormat = WebMessageFormat.Json, ResponseFormat = WebMessageFormat.Json)]
    public void DeidentificationPOSTRequest(Stream inputStream)

        [WebInvoke(Method = "POST", UriTemplate = "/deidentifysync", BodyStyle =
        WebMessageBodyStyle.Bare, RequestFormat = WebMessageFormat.Json, ResponseFormat  =
        WebMessageFormat.Json)]
    public DeidentificationJobStatus DeidentificationSynchronousPOSTRequest(Stream inputStream)

        [WebGet(UriTemplate = "/anonymizationreport/{JobID}", ResponseFormat =
        WebMessageFormat.Json)]
    public Stream GetDeidentificationReport(string JobID);


        [WebGet(UriTemplate = "/status", ResponseFormat = WebMessageFormat.Json)]
    public DeidentificationJobStatus[] GetCurrentStatus();

        [WebGet(UriTemplate = "/jobreport/{JobID}", ResponseFormat = WebMessageFormat.Json)]
    public Stream GetJobStatus(string JobID);

        [WebGet(UriTemplate =
        "/studyreport/jobs/{JobID}/databases/{Database}/patients/{PatientID}/studies/{StudyID}",
        ResponseFormat = WebMessageFormat.Json)]
    public Stream GetStudyReport(string JobID, string Database, string PatientID, string StudyID);

        [WebGet(UriTemplate = "/context", ResponseFormat = WebMessageFormat.Json)]
    public ContextInfo[] GetContextInfo();

        [WebInvoke(Method = "DELETE", UriTemplate = "/context/{JobID}")]
    public void DeleteContext(string JobID);

}

public class DeIdentificationRequest
{
    public StudyEntry[] StudyEntries { get; set; }
    public int ID { get; set; }
    public string ProfileID { get; set; }
    public DicomDestination RemoteDicomDestination { get; set; }
    public string StatusCallbackURI { get; set; }
    public Dictionary<string, Dictionary<string, string>> SubstitionValues { get; set; }
    public int IndexSeed;
}

public class StudyEntry
{
    public string StudyInstanceUID { get; set; }
    public string Database { get; set; }
}

public class DeidentificationJobStatus
```

© Hyland Software, Inc. All rights reserved.

```
{
    public int ID { get; set; }
    public int Successful { get; set; }
    public int Failed { get; set; }
    public int PercentCompletion { get; set; }
    public long ElapsedMilliseconds { get; set; }
}


public class DeidentificationReport
{
    public DeidentificationReportEntry[] Entries { get; set; }
}


public class DeidentificationReportEntry
{
    public string Database { get; set; }
    public string OriginalPatientID { get; set; }
    public string OriginalAccessionNumber { get; set; }
    public string OriginalStudyInstanceUID { get; set; }
    public string AnonymizedPatientID { get; set; }
    public string AnonymizedAccessionNumber { get; set; }
    public string AnonymizedStudyInstanceUID { get; set; }

}

public class DeidentificationStatusReport
{
    public DeidentificationStatusReportEntry[] Entries { get; set; }
}

public class DeidentificationStatusReportEntry
{
    public string Identifier { get; set; }
    public string Status { get; set; }
    public string Database { get; set; }
    public string TimeStamp { get; set; }
    public int InstanceCount { get; set; }
    public string ErrorMessage { get; set; }
}

public class DicomDestination
{
    public string LocalAETitle { get; set; }
    public string RemoteAETitle { get; set; }
    public string RemoteHost { get; set; }
    public int RemotePort { get; set; }
}

public class ContextInfo
{
    public int ID { get; set; }
    public long Size { get; set; }
    public string CreationDateTime { get; set; }
    public string LastUpdateDateTime { get; set; }
}
```

© Hyland Software, Inc. All rights reserved.

Annex B – Expanded API

```
    [WebInvoke(Method = "POST", UriTemplate = "/analytics", BodyStyle = WebMessageBodyStyle.Bare,
    RequestFormat = WebMessageFormat.Json, ResponseFormat = WebMessageFormat.Json)]
public void AnalyticsPOSTRequest(Stream inputStream)

   [WebInvoke(Method = "POST", UriTemplate = "/analyticssync",
   BodyStyle = WebMessageBodyStyle.Bare, RequestFormat = WebMessageFormat.Json,
   ResponseFormat = WebMessageFormat.Json)]
public DeidentificationJobStatus AnalyticsSynchronousPOSTRequest(Stream inputStream)


public class AnalyticsRequest
{
        public string Type { get; set; }
        public StudyEntry[] StudyEntries { get; set; }
        public int ID { get; set; }
        public string ProfileID { get; set; }
        public string TargetDirectory { get; set; }
        public string Collection { get; set; }
        public DicomDestination RemoteDicomDestination { get; set; }
        public string STOWURI { get; set; }
        public string StatusCallbackURI { get; set; }
        public Dictionary<string, Dictionary<string, string>> SubstitionValues { get;    set; }
        public int IndexSeed;
    }
```

Deidentification Service FAQs

Deidentified Person Names values, what is the best approach?

If BulkMode is true, there is only one option, which is taking advantage of the PseudonymPattern property. However if BulkMode is false, which is the default, there are two distinct modes. If PseudonymPattern is set, than the behavior is identical to BulkMode, an obvious test template is being used, indicating synthetic data. If PseudonymPattern is not specified as a property, then a dictionary of Person Names will be used, that will be randomly assembled. This data is more realistic and in a way easier to index. There is no risk for it to match a real person with its original studies by design. However the dictionary values can obviously match real person names, and this is by design. Some users were uncomfortable with this approach, though it does guarantee that the studies cannot be traced back to the original patients or physicians. It all depends if fake synthetic data is ok, and gives peace of mind, or if a more realistic data set (though still decoupled from the originals) is needed.

```
<setting name="PseudonymPattern" serializeAs="String">
        <value>TEST{0}^ANONYMOUS^^^</value>
</setting>
```

What is the speed of this process?

The answer will unfortunately disappoint. It really does depend on many factors. Besides the obvious existing infrastructure, the deidentification service is running against, the main factors that determine the duration of a deidentification processing batch are the size of the batch and the types of the studies. In

© Hyland Software, Inc. All rights reserved.

general, the APIs used by this process, are scalable and performant. It will depend if the data will have to be staged to primary tier from an archiving device, etc.

However, the type of the studies is also very important. A chest CR or DX will have only have a few DICOM instances, while MR and CTS can have thousands of DICOM instances.

Then there is the size of the batch, if the mode used is BulkMode = false, the default. This is because the implementation uses a deidentification context. Please think of the deidentification context as a collection and mapping of DICOM UIDs between original and deidentified data sets. Using a context will assure that for a specific batch studies of the same patients, series of the same study, and instances of the same series will be consistently mapped, not just from a keys perspective however all other deidentified data values. As explained in the main document, the deidentification context can grow very large. The default implementation and configuration will save and reload the context for every single study. In most cases, exceptions are rare in the case of deidentification operations, so we added a new way to control the frequency of the save. The larger the batch size, and the larger the study type size, the longer the context save and reload operations will take.

This is not the case in BulkMode = true.

Back to BulkMode = false.

The ContextSaveFrequency will save the deidentification context only every n entries, value specified as part of the <Hyland.Nil.Deidentify.Properties.Settings> section. The default value is 1, and in line with the initial default behavior. Of course this applies for BulkMode = false. The deidentification context is also saved when the batch of entries for deidentification has completed. Using a larger value for this property can significantly reduce the duration of the deidentification operation.

The only tradeoff is if the processing of the batches gets interrupted. In that case, the context saved on disk might not be in synch with the data in the output directory. This situation is rare, and the efficiency gains are significant in the case of large batches. To give you a more concrete example, if the batch size is 10,000 and a ContextSaveFrequency value of 50 is used, this will have the context persistence call performed only 200 times rather than 10,000 times. As the deidentification context grows – also dependent on the study type and size (number of DICOM instance) – the save operation becomes significant in duration (over a second), so the use of this configuration property is strongly encouraged for large batches and once the operators have confidence in the quality of their DICOM data. No exceptions will be thrown if no DICOM violations will be detected.

```
<setting name="ContextSaveFrequency" serializeAs="String">
        <value>50</value>
</setting>
```

You have to understand your data and your use cases before estimating the total time it will take to deidentify your intended research data set and this will also allow you to design the best approach for using optimal batch sizes. Running a smaller test batch is always recommended for estimating total duration and the quality of the data set.

Deidentification context and its continuity

Every submission and job will have its own deidentification context. The deidentification context is used as a set of collections and mappings of DICOM UIDs and data elements between original and deidentified data sets.

© Hyland Software, Inc. All rights reserved.

The IndexSeed property plays a very important role, since it will represent the starting value for the iterative assignment of new keys and values (for the tags this applies to). For concurrent submissions, programmatic or via the watch directory, the IndexSeed value in the properties will be used. In this case, the submissions are deemed independent and potential overlaps in deidentified values should not be an issue. Also, please remember that the deidentification context can grow large and therefore influence the overall deidentification duration, especially if the default value of 1 is being used for the ContextSaveFrequency.

The jobID is important, it can be reused for subsequent submissions, both programmatically and via the watch directory (by using the xxxx_ prefix file name convention). If the same study is submitted as part of a batch with the same jobID, it will be deidentified the same way. Reusing a jobID makes sense in the case of data that belongs to the same patient or study, or in the case of interrupted batches. If the history property is used, not set to be ignored, one has to be careful in the case of interrupted batches and increase the IndexSeed value. Interrupting batches should be generally avoided.

Determining what value to increase the IndexSeed to is more complex. One can open the context file and figure out the highest value used. This is the safest way. The number of studies, does directly affect the IndexSeed choice however it all depends on other data elements being present in the dataset. For example 10,000 studies might need an adjustment to 15,000, 20,000 or higher of the IndexSeed value.

In the case of BulkMode = true, the IndexSeed can be increased by the number of studies processed * 10 + a small buffer or roundup. 10 is currently a constant and works well for patient batches (BulkMode works at the patient aggregation level) that have a moderate number of studies.

Deidentifying large datasets

It is hard to recommend an optimal batch size. Similar to assessing performance and duration of a deidentification batch, it depends on the infrastructure, and the type of the studies in the batch. For studies with fewer DICOM instances, like CR, DX, MG, the batch sizes can be larger. More caution needs to be applied for CT and MR study types.

A 100,000 data set of chest X-Ray can probably be split into 4 or 5 batches, 25000 or 20000.

A 100,000 batch of CT studies will probably require a lower batch size, 5000 to 10,000.

Regardless, we recommend the following. Each batch should have its own job id. After each batch, the IndexSeed for the next batch will need to be set to a higher value. Determining this higher value is currently a manual step, examining the context is the best approach if BulkMode = false. For BulkMode = true, the size of the batch times 10 plus a padding value is a good computation.

The other important feature is a larger value for the ContentSaveFrequency property (BulkMode = false). This will speed up the processing of a batch.

In the future, we might provide a simple utility to analyze a deidentification context.

If the metadata is not used, for example the deidentified data set will be used only for training of visualization algorithms and deep learning algorithms, the same IndexSeed can be used. However, in this case, there will be overlaps for PatientIDs, Accession Numbers and the Person Name data elements. If the entire data set will be stored into a DICOM repository, we recommend paying attention to the IndexSeed value. Otherwise, if only the pixel data and pixel data parameters will be used for research the

© Hyland Software, Inc. All rights reserved.

demographic integrity is not necessary. The DICOM instances will still be correctly grouped at the study level.

In general, we recommend that the data set be grouped/order by patient, even if the granularity sought is study level. This will not split studies of the same patient over multiple batches, and assure a correct grouping.

Which tags can be deidentified?

The implementation follows the DICOM standard, please refer to PS 3.15 Annex E, Attribute Confidentiality Profiles.

While we do not encourage exceptions to the deidentification profiles, the deidentification implementation allows for exceptions, which can be configured after consulting with Hyland.

Deidentified Dates

The dates for the values being replaced will be over 100 years in the future. These should store without issues in the Acuo VNA, and are searchable from the Admin Portal, the MMC and the SQL Management Studio. In the MMC, if no results are being retrieved, the study date begin and end values shall be used.

Infrastructure prerequisites

The deidentification operation is not very resource intensive. If a large number of studies are being deidentified and the system uses an archiving tier, the cache use could increase non-linearly and outside the usual operation parameters.

The other main consideration is available storage space will be needed either on the partition used as output directory or the DICOM repository the studies are being sent to.

Research Module in Admin Portal

We believe that the research module in the Admin Portal is a great tool for administrators who are constantly bombarded by requests for deidentified data. Several jobs can be submitted in parallel to the Dedidentification service, and these jobs will all use the same IndexSeed value. The jobs should be considered separate and no additional correlation should be expected. If jobs correspond to batches of the same data set, the approach described earlier in the FAQ document is recommended.

How to deidentify Specific Dicom Tags

There are 2 ways of deidentifying specific tags

1. Using AnonymizationActions
2. Using AnonymizationExtensions

1. Using AnonymizationActions

In Application Configuration file (AcuoDeidentificationService.exe.config), there is a scope to define several parameters for DeidentificationProfile. And multiple DeidentificationProfiles can be defined under DeidentificationProfileConfigSection.
In below screenshot of a configuration file 2 Profiles (Deidentification Configurations) are defined –

© Hyland Software, Inc. All rights reserved.

ResearchGroup1 & PartnerProfile2.
PartnetProfile2 consists a profile parameter named **AnonymizationActions** (as shown in below screenshot).

```
    <add key="DeidentificationSCPDirectory" value="D:/Acuo/deidentification/dicomdeid" />
    <add key="DeidentificationSCPCheckFrequency" value="60" />
    <add key="ReidentificationSCP" value="REIDAE:1046" />
    <add key="ReidentificationSCPDirectory" value="D:/Acuo/deidentification/dicomreid" />
    <add key="ReidentificationSCPCheckFrequency" value="60" />
  </appSettings>
  <DeidentificationProfileConfigSection>
    <DeidentificationProfiles>
      <DeidentificationProfile ProfileId="ResearchGroup1" PseudonymPattern="Test1{0}^Test2^^^" DeidentificationEpochs="1960" TimeShiftTag="0x00100030" />
      <DeidentificationProfile ProfileId="PartnerProfile2" AnonymizationActions="[{&quot;Tag&quot;: &quot;(0008,0050)&quot;,&quot;BasicProfile&quot;: &quot;Z&quot;}]" />
    </DeidentificationProfiles>
  </DeidentificationProfileConfigSection>
  <applicationSettings>
    <Hyland.Nil.Deidentify.Properties.Settings>
      <setting name="ScrubberTrim" serializeAs="String">
        <value>()[]{}&lt;&gt;,.:;'"|\+-#</value>
      </setting>
      <setting name="ScrubberDefaultAllowed" serializeAs="String">
        <value />
      </setting>
```

- Deidentification Profiles

Support for multiple deidentification profiles has been provided in the configuration file (AcuoDeidentificationService.exe.config) to support more than one research integration (probable requirements in large institutions). Idea is to provide different configurations per deployment identified by a profile id, which is a string.

```
<DeidentificationProfileConfigSection>
        <DeidentificationProfiles>
                <DeidentificationProfile ProfileId="ResearchGroup1" DeidentificationEpochs="1960"
                TimeShiftTag="0x00100030"/>
                <DeidentificationProfile ProfileId="PartnerProfile2"
                AnonymizationActions="[{&quot;Tag&quot;:
                &quot;(0008,0050)&quot;,&quot;BasicProfile&quot;: &quot;Z&quot;}]"/>
        </DeidentificationProfiles>
</DeidentificationProfileConfigSection>
```

The above config file (AcuoDeidentificationService.exe.config) segment shows the definition of two deidentification profiles, identified by profile ids – ResearchGroup1 and PartnerProfile2. There are list of properties that can be defined per profile. If a property is not defined in a profile, it will be defined by the default property.

The ProfileId is a string, and in most cases, it will be submitted via a programmatic RESTful POST request, however it can also be specified as part of the name of a file dropped into the watch directory (as the second component after the Job ID).

- How to define AnonymizationActions

In some situations, the users want to be in full control of the de-identification actions, decoupling these from the actions defined by the DICOM standard's Attribute Confidentiality Profiles.
If property AnonymizationActions is not empty or null, only the actions defined by this property will be applied to the DICOM instances.
For example –
AnonymizationActions="[{"Tag": "(0008,0050)","BasicProfile": "Z"}]"

To make this easier to read, we have to substitute " with ".
This simplified example means that only tag 0x00080050 (Accession Number) will be applied action 'Z', which is defined as Z - replace with a zero length value, or a non-zero length value that may be a dummy

© Hyland Software, Inc. All rights reserved.

value and consistent with the VR.
AnonymizationActions="[{“Tag”: “(0008,0050)”,”BasicProfile”: “Z”}]"

Z is a **DeIdentification Action Code** here, and more details are mentioned here.
Any of the listed codes can be used (as per requirement) in the BasicProfile value.

2. Using AnonymizationExtensions

If ProfileId mentioned in the DeIdentification request (or, in the input file of inputwatch mechanism) is not available/defined in the application configuration file (AcuoDeidentificationService.exe.config), the system applies **Default Profile** for any Deidentification Request.

The AnonymizationExtensions are defined under applicationSettings and thereafter Hyland.Nil.Deidentify.Properties.Settings in the configuration file (as shown in the below screenshot).

```
  </DeidentificationProfiles>
</DeidentificationProfileConfigSection>
<applicationSettings>
  <Hyland.Nil.Deidentify.Properties.Settings>
    <setting name="ScrubberTrim" serializeAs="String">
      <value>()[]{}&lt;&gt;,.;:'"|\+-#</value>
    </setting>
    <setting name="ScrubberDefaultAllowed" serializeAs="String">
      <value />
    </setting>
    <setting name="ScrubberDefaultUntouched" serializeAs="Xml">
      <value>
        <ArrayOfString xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w
          <string>SeriesDescription</string>
          <string>ClinicalTrialSeriesDescription</string>
        </ArrayOfString>
      </value>
    </setting>
    <setting name="PseudoidFormat" serializeAs="String">
      <value>SSG{0}</value>
    </setting>
    <setting name="PseudonymPattern" serializeAs="String">
      <value>TEST{0}^ANONYMOUS^^^</value>
    </setting>
    <setting name="AnonymizationExtensions" serializeAs="String">
      <value>
        [
            {
            "Tag": "(0008,0050)",
            "BasicProfile": "Z"
            },
            {
            "Tag": "(0010,0010)",
            "BasicProfile": "Z"
            },
            {
            "Tag": "(0010,0020)",
            "BasicProfile": "Z"
            }
        ]
      </value>
    </setting>
    <setting name="ExtendedTags" serializeAs="String">
      <value>
        {"0xFFFEe000":[{"tag":"0x20050010","name":"Private Creator", "vr":"SH"}]}
      </value>
    </setting>
```

- Steps to Activate/Enforce Anonymization Actions

  There are 2 ways to enforce this feature:
  (1) By using Inputwatch directory
  (2) By RestAPI call

© Hyland Software, Inc. All rights reserved.

Using Inputwatch directory

(1) InputWatchDirectory is a drop directory for the input files that contain studies to be deidentified. InputWatchDirectory location must be provided in the config file (AcuoDeIdentificationService.exe.config).

```
<add key="DecoupledMode" value="false" />
<add key="DeidentificationMaxDOP" value="1" />
<add key="BulkMode" value="false" />
<add key="InputWatchDirectory" value="D:/Acuo/deidentification/inputwatch" />
<add key="DeidentificationDirectory" value="D:/Acuo/deidentification/output" />
<add key="ReidentificationDirectory" value="D:/Acuo/deidentification/output/reidentify" />
<add key="ContextDirectory" value="D:/Acuo/deidentification/context" />
```

(2) The ideal naming convention of a input file must be like – <JobId>_<ProfileId>_<date>.txt
That means, if one imaginary JobId is 20300 and intended Profile to be applicable in a DeIdentification Action is PartnerProfile2, then the input file name would be **20300_PartnerProfile2_07032024.txt**.

- **JobId** is an integer value, which can be used in different other correspondence of that particular transaction.
- **ProfileId** is a string value. All the other Information about Profiles are described here.

(**N.B.** –
The input file consists of a collection of lines, each line identifying a DICOM study. There exists multiple ways to identify a study. Each line is a list of fields, separated by the '|' character. The first field is the type of identification, which is one of 'OID', 'ACC', 'MRNSTD' and 'MRNBLK' (for the bulk option))

(3) The input file should be placed inside the InputWatchDirectory. The output images will placed in the output directory (DeidentificationDirectory as defined in the Configuration file)

```
<add key="DecoupledMode" value="false" />
<add key="DeidentificationMaxDOP" value="1" />
<add key="BulkMode" value="false" />
<add key="InputWatchDirectory" value="D:/Acuo/deidentification/inputwatch" />
<add key="DeidentificationDirectory" value="D:/Acuo/deidentification/output" />
<add key="ReidentificationDirectory" value="D:/Acuo/deidentification/output/reidentify" />
<add key="ContextDirectory" value="D:/Acuo/deidentification/context" />
<add key="Part10WebServiceEndpoint" value="http://localhost:8081/AcuoREST/genp10" />
```

Rest-API Call

(1) User can use any RestAPIClient to avail this feature.
(2) The Rest Service Address is defined in the configuration file –

```
<add key="ContextDirectory" value="D:/Acuo/deidentification/context" />
<add key="Part10WebServiceEndpoint" value="http://localhost:8081/AcuoREST/genp10" />
<add key="Part10Directory" value="D:/Acuo/deidentification/part10" />
<add key="HistoryDirectory" value="D:/Acuo/deidentification/history" />
<add key="IgnoreHistory" value="false" />
<add key="AnonymizationRecordDirectory" value="D:/Acuo/deidentification/anonymizationrecord" />
<add key="MetadataDirectory" value="D:/Acuo/deidentification/metadata" />
<add key="AcuoAuthorizationGroups" value="Acuo Users,Acuo Power Users,Acuo Administrators" />
<add key="RestServiceAddress" value="http://0.0.0.0:8099/AcuoDeidentification" />
<add key="RestServiceAuthAddress" value="http://0.0.0.0:8399/AcuoDeidentification" />
<add key="RestTokenAuth1" value="false" />
```

The Rest Service URI for deidentification Request would be like this –
<RestServiceAddress>/deidentify
e.g. –

© Hyland Software, Inc. All rights reserved.

http://localhost:8099/AcuoDeidentification/deidentify

In the POST call the Rest-API JSON body structure (sample) looks like as shown below –


```
{
  "StudyEntries" :
  [
    {
      "StudyInstanceUID": "1.3.46.670589.11.0.0.11.4.2.0.1.5.5864.2004031716583779062",
      "Database": "dicomdb1",
      "Series": []
    }
  ],
  "ID" : 20300,
  "ProfileID" : "ProfileTest2",
  "RemoteDicomDestination" : null,
  "StatusCallbackURI" : null,
  "SubstitionValues": {"00100020":{"Original_PatientId_15454564":"SBST_PatId_1234","ID5678":"SBST5678"}}
}
```


(3) The output images will be placed in the output directory (DeidentificationDirectory as defined in the Configuration file)

Deidentification Action Codes

Action Codes	Description
D	replace with a non-zero length value that may be a dummy value and consistent with the VR
Z	replace with a zero length value, or a non-zero length value that may be a dummy value and consistent with the VR
X	remove
K	keep (unchanged for non-Sequence Attributes, cleaned for Sequences)
C	clean, that is replace with values of similar meaning known not to contain identifying information and consistent with the VR
U	replace with a non-zero length UID that is internally consistent within a set of Instances
Z/D	Z unless D is required to maintain IOD conformance (Type 2 versus Type 1)
X/Z	X unless Z is required to maintain IOD conformance (Type 3 versus Type 2)
X/D	X unless D is required to maintain IOD conformance (Type 3 versus Type 1)
X/Z/D	X unless Z or D is required to maintain IOD conformance (Type 3 versus Type 2 versus Type 1)
X/Z/U*	X unless Z or replacement of contained instance UIDs (U) is required to maintain IOD conformance (Type 3 versus Type 2 versus Type 1 sequences containing UID references)

***Source** - DICOM PS3.15 - Security and System Management Profiles

External Reference

- "DICOM standard's Attribute Confidentiality Profiles" will be available here – https://dicom.nema.org/medical/dicom/current/output/html/part15.html#sect_E.1

© Hyland Software, Inc. All rights reserved.